

PRD (Product Requirements Document)

Proje Adı

MicroBank – Mikroservis Tabanlı Temel Bankacılık ve Kredi Başvuru Sistemi

1. Proje Özeti

Amaç

MicroBank, mikroservis mimarisi kullanılarak geliştirilecek modern bir dijital bankacılık uygulamasıdır. Sistem, kullanıcıların güvenli şekilde hesap oluşturmasını, banka hesaplarını yönetmesini, para transferi yapmasını ve kredi başvurularında bulunmasını sağlar.

Kredi değerlendirme süreci senkron çalışmak yerine mesajlaşma altyapısı üzerinden asenkron olarak gerçekleştirilir. Böylece gerçek bankacılık sistemlerinde kullanılan dağıtık mimari yaklaşımı simüle edilir.

Projenin en önemli amacı yalnızca çalışan bir bankacılık uygulaması geliştirmek değil, aynı zamanda profesyonel seviyede test otomasyonu, API testleri, UI testleri ve performans testlerini de içeren tam kapsamlı bir yazılım geliştirme süreci oluşturmaktır.

2. Proje Hedefleri

- Mikroservis mimarisini uygulamak
 - JWT tabanlı güvenli kimlik doğrulama geliştirmek
 - RESTful API tasarlamak
 - Transaction yönetimini doğru şekilde gerçekleştirmek
 - Asenkron Event Driven Architecture kullanmak
 - Redis ile cache yönetimi yapmak
 - PostgreSQL üzerinde ilişkisel veri modeli oluşturmak
 - Docker ile servisleri containerize etmek
 - CI/CD pipeline oluşturmak
 - Test otomasyonunu profesyonel seviyeye taşımak
-

3. Hedef Kullanıcılar

Bireysel Banka Müşterisi

Yapabilecekleri

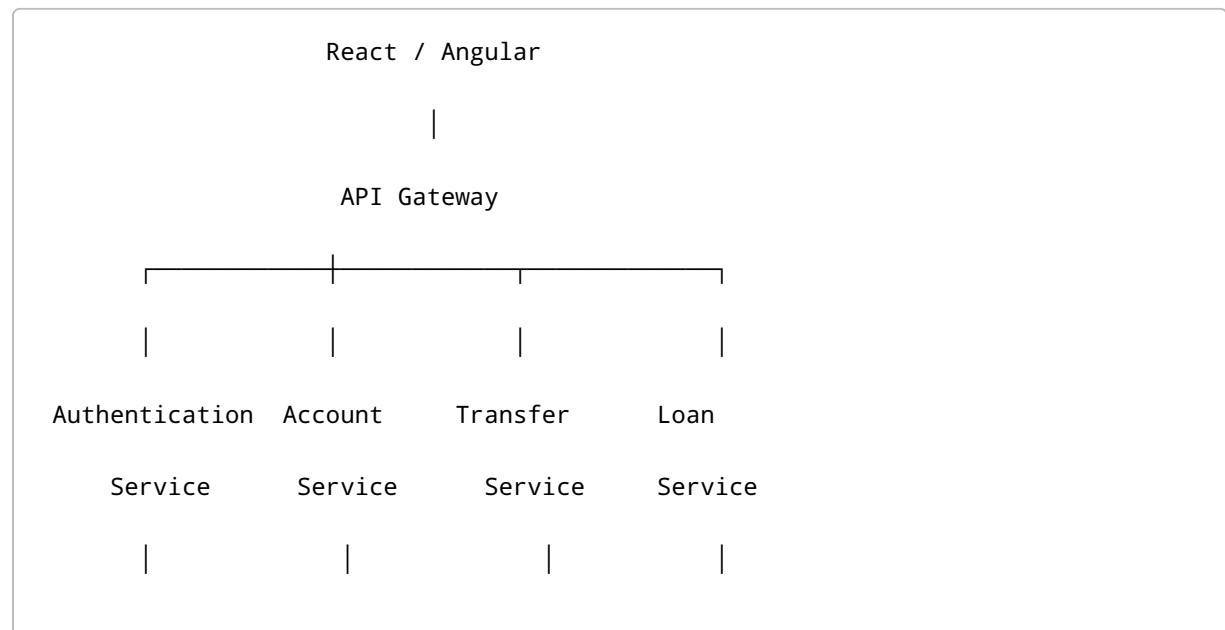
- Kayıt olmak
- Giriş yapmak
- Hesap açmak
- Bakiye görüntülemek
- Para transfer etmek
- Kredi başvurusu yapmak
- Başvuru durumunu takip etmek

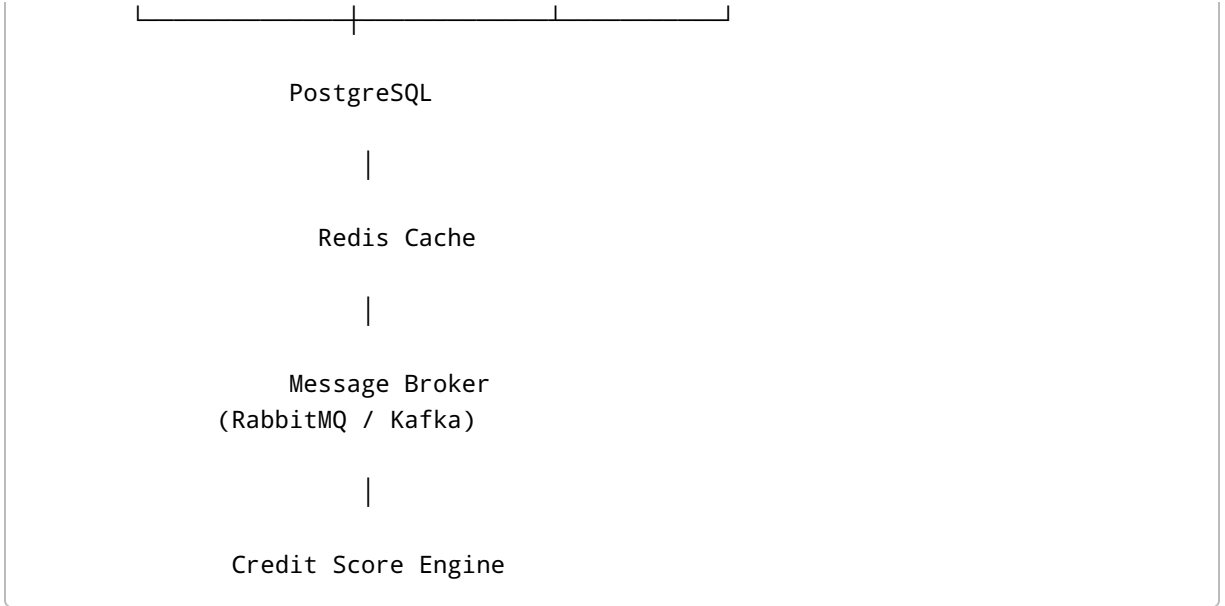
Sistem Yöneticisi

Yapabilecekleri

- Kullanıcıları görüntülemek
- Hesapları görüntülemek
- Transfer geçmişini incelemek
- Kredi başvurularını görüntülemek
- Sistem loglarını takip etmek

4. Proje Mimarisi





5. Kullanılacak Teknolojiler

Backend

- Java Spring Boot
- Spring Security
- Spring Data JPA
- Spring Cloud
- Maven
- Hibernate

Alternatif

- .NET Core

Frontend

- React
- TypeScript
- Material UI

Alternatif

- Angular

Database

- PostgreSQL
 - Redis
-

Messaging

- RabbitMQ

Alternatif

- Apache Kafka
-

Container

- Docker
 - Docker Compose
-

Authentication

- JWT
 - BCrypt
-

Test

Backend

- JUnit
- Mockito
- RestAssured

UI

- Selenium
- Playwright

API

- Postman
- Newman

Performance

- Apache JMeter

CI/CD

- GitHub Actions
-

6. Mikroservisler

Authentication Service

Görevleri

- Kullanıcı kayıt
- Login
- JWT üretme
- Token doğrulama

Endpointler

POST /auth/register

POST /auth/login

GET /auth/me

Account Service

Görevleri

- Hesap oluşturma
- Hesap listeleme
- Bakiye görüntüleme
- Hesap bilgileri

Endpointler

POST /accounts

GET /accounts

GET /accounts/{id}

GET /accounts/balance/{id}

Transfer Service

Görevleri

- Havale
- EFT
- Transaction yönetimi
- İşlem geçmişi

Endpointler

POST /transfer

GET /transfer/history

Loan Service

Görevleri

- Kredi başvurusu
- Başvuru durumu
- Skor değerlendirmesi

Endpointler

POST /loan/apply

GET /loan/{id}

GET /loan/history

7. Kredi Skorlama Motoru

Başvuru yapıldığında:

Loan Apply

↓

RabbitMQ

↓

Credit Score Worker

↓

Random Score

↓

Gelir Analizi

↓

Limit Hesabı

↓

Onay / Red

↓

Database Güncelle

Örnek kurallar

Skor < 500

RED

500-700

50.000 TL

700-850

150.000 TL

850+

300.000 TL

8. Veritabanı Tasarımı

Customer

Alan	Tip
id	UUID
firstName	varchar
lastName	varchar
email	varchar
password	varchar
createdAt	timestamp

Account

Alan	Tip
id	UUID
customerId	UUID
accountType	enum
iban	varchar
balance	decimal
status	enum

Transfer

Alan	Tip
id	UUID
senderId	UUID
receiverId	UUID
amount	decimal
createdAt	timestamp

LoanApplication

Alan	Tip
id	UUID
customerId	UUID
income	decimal
score	int
limit	decimal
status	enum
createdAt	timestamp

9. İş Akışları

Kullanıcı Kayıt

Register

↓

Validation

↓

Password Hash

↓

Database

↓

JWT

Para Transferi

Transfer

↓

Validation

↓

Balance Check

↓

Database Transaction

↓

Sender Balance -

↓

Receiver Balance +

↓

Commit

↓

History

Rollback durumları

- Eksik bakiye
- Hesap bulunamadı

- Database hatası
-

Kredi Başvurusu

Loan Apply

↓

RabbitMQ

↓

Worker

↓

Random Credit Score

↓

Income Check

↓

Decision

↓

Database

↓

Frontend Notification

10. Frontend Sayfaları

Authentication

- Login
 - Register
-

Dashboard

- Toplam bakiye
 - Hesaplar
 - Son işlemler
-

Hesap Yönetimi

- Hesap oluştur
 - Hesap detay
-

Transfer

- IBAN seç
 - Alıcı seç
 - Tutar gir
 - Onay
-

Loan

- Gelir bilgisi
 - Başvuru
 - Sonuç ekranı
-

Admin

- Kullanıcılar
 - Transferler
 - Krediler
-

11. Güvenlik Gereksinimleri

- JWT Authentication
- BCrypt Password
- HTTPS
- Input Validation
- SQL Injection koruması
- XSS koruması
- CORS yönetimi

- Rate Limiting
 - Role Based Authorization
-

12. Test Stratejisi

Unit Test

Kapsam

- Service katmanı
- Utility sınıfları
- Mapper'lar

Hedef

%80+ Code Coverage

Integration Test

Test edilecekler

- PostgreSQL
 - Redis
 - RabbitMQ
 - REST API
-

API Testleri

Araç

RestAssured

Pozitif Senaryolar

- Başarılı login
- Hesap oluşturma
- Başarılı transfer
- Başarılı kredi başvurusu

Negatif Senaryolar

- Eksik token

- Geçersiz JWT
- Negatif transfer
- Hesap bulunamadı
- Limit aşımı
- Eksik parametre

Boundary Value Analysis

Transfer

-100 TL
0 TL
1 TL
999999 TL
999999999 TL

Kredi

Gelir
0
1
9999
10000
10001

UI Test

Framework

Playwright veya Selenium

POM Yapısı

pages

LoginPage

DashboardPage

LoanPage

TransferPage

E2E Senaryosu

Login

↓

Dashboard

↓

Loan Page

↓

Form Doldur

↓

Başvur

↓

Sonucu Doğrula

Performance Test

Araç

Apache JMeter

Senaryo

1000 Concurrent User

↓

POST /transfer

↓

Response Time

↓

Throughput

↓

Error Rate

Başarı Kriterleri

- Ortalama Response Time < 500 ms
- Error Rate < %2
- Availability > %99

13. CI/CD

GitHub Flow

Developer

↓

Pull Request

↓

GitHub Actions

↓

Unit Test

↓

Integration Test

↓

API Test

↓

Docker Build

↓

Deploy

14. Docker Yapısı

docker-compose

postgres

redis

rabbitmq

auth-service

account-service

transfer-service

loan-service

frontend

gateway

15. Proje Klasör Yapısı

```
microbank/  
  
frontend/  
  
gateway/  
  
auth-service/  
  
account-service/  
  
transfer-service/  
  
loan-service/  
  
shared-library/  
  
docker/  
  
performance-tests/  
  
api-tests/  
  
ui-tests/  
  
docs/  
  
README.md
```

16. Başarı Kriterleri

Fonksiyonel

- Kullanıcı kayıt olabilir.
- Güvenli giriş yapılabilir.
- JWT ile yetkilendirme çalışır.
- Hesap açılabilir.
- Bakiye görüntülenebilir.
- Transfer işlemleri başarılı şekilde tamamlanır.
- Kredi başvurusu asenkron değerlendirilir.
- Başvuru sonucu kullanıcıya gösterilir.

Teknik

- Mikroservis mimarisi uygulanmıştır.
 - Docker Compose ile tüm servisler ayağa kalkar.
 - PostgreSQL ve Redis entegre çalışır.
 - RabbitMQ üzerinden event tabanlı iletişim sağlanır.
 - API dokümantasyonu (Swagger/OpenAPI) sunulur.
 - Unit, Integration, API, UI ve Performance testleri başarıyla çalışır.
 - GitHub Actions üzerinden otomatik test ve build süreçleri yürütülür.
-

17. Gelecek Sürümler (Roadmap)

v1.0

- Kullanıcı yönetimi
- Hesap yönetimi
- Para transferi
- Kredi başvurusu

v1.1

- İşlem bildirimleri (E-posta/SMS simülasyonu)
- Döviz hesapları
- Hesap hareketleri filtreleme

v1.2

- Mobil uygulama
- QR ile para transferi
- Yapay zekâ destekli kredi skorlama
- Gerçek banka API entegrasyonu (sandbox)
- Prometheus + Grafana ile izleme ve metrik toplama